Build a reliable job-feed ingestion service

Artha.link · Full Stack Developer · 1–3 years · TypeScript + MongoDB

Purpose and time budget

Build a small service that accepts job events, processes them asynchronously and exposes the latest job state. We care most about how you reason about correctness, failures and scale. A simple implementation you can explain is better evidence than a large feature set.

Spend up to **8 focused hours**, submitted within **72 hours of receiving the assignment**. Record actual time and unfinished items. These are the default assessment windows; agree any scheduling adjustment before starting. Design notes may describe production features you do not implement. There is no UI requirement and no paid cloud or AI API requirement.

AI assistance is expected for at least one meaningful part of the work, such as implementation, test design or design critique. You remain responsible for every submitted claim and line of code. Free tools are sufficient; we do not score tool brand or subscription level. If you cannot access an AI tool, tell us before starting so an equivalent arrangement can be agreed.

Required stack and scope

- Use Node.js 22 or newer and TypeScript with strict checking for the API, worker, domain logic, tests and demo/load scripts. Configuration and shell wrappers can use their normal formats.
- Use MongoDB for durable event/work state and current job state. Provide Docker Compose for local setup. A replica set is allowed if your design needs transactions.
- Choose any TypeScript server/test libraries. Avoid adding Redis, Kafka or a managed queue to the runnable core; discuss those alternatives in the design document if useful.
- Provide a Git repository that runs locally. A hosted URL or a frontend is optional and carries no extra points by itself.
- Use only the supplied synthetic data and your own synthetic test data. No access to a production platform is needed.

Event contract

Implement `POST /events` for one JSON event at a time:

```
{
  "tenantId": "tenant-a",
  "sourceId": "main",
  "eventId": "event-101",
  "externalJobId": "alpha",
  "version": 1,
  "operation": "upsert",
  "payload": {
    "title": "Full Stack Developer",
    "company": "Example Labs",
    "location": "Surat",
    "experienceMin": 1,
    "experienceMax": 3,
    "applyUrl": "https://example.test/jobs/alpha",
    "skills": [" TypeScript ", "MongoDB", "typescript"]
  }
}
```

The string identifiers `tenantId`, `sourceId`, `eventId` and `externalJobId` must be nonblank; reject surrounding whitespace instead of silently changing identifiers. `version` is a positive safe integer.

`operation` is `upsert` or `archive`. An upsert requires the shown payload fields. Title, company and location must be nonblank after trimming. Experience values are integers from 0 to 50 with minimum no greater than maximum. The URL must use HTTPS. Skills are an array of nonblank strings. Trim display strings and normalize skills to trimmed lowercase values with duplicates removed, preserving first occurrence order. An archive omits `payload`.

A job is identified by `(tenantId, sourceId, externalJobId)`. An event is identified by `(tenantId, sourceId, eventId)`. The source guarantees that a job version describes one semantic change; equivalent retransmissions can have a new event ID. Conflicting content for the same job version is outside the core fixture contract: document how you would handle it.

Observable behavior

1. **Durable acceptance:** return `202` only after the event is durably recorded for processing. Do not run the whole job synchronously inside the request. A crash after acceptance must not silently lose work.
2. **Replay safety:** the same event identity and same parsed JSON document returns `200` and creates no second work item. Ignore object-key order when comparing JSON; array order remains significant. Reusing that event identity with different content returns `409`. Invalid events return `400` and do not reserve an event identity. A corrected request may reuse the rejected ID. For valid input, apply replay checks before treating a request as new work.
3. **Version safety:** the greatest successfully processed version determines current job state, regardless of delivery or worker completion order. Lower/equivalent versions cannot overwrite it. An archive preserves a versioned tombstone so delayed old upserts cannot reactivate the job. An archive can arrive before any upsert.
4. **Isolation:** matching IDs in another tenant or source are separate events/jobs. Every read must be scoped by tenant; optionally filter by source. For this exercise, tenant ID is a trusted demo input. Explain production authentication and tenant authorization in your design.
5. **Background processing:** show at least two worker instances or loops competing for work. Claims, completion and recovery must remain safe under concurrency. Show that abandoned work can be recovered after a worker stops. In-process memory alone is not durable recovery.
6. **Provider failures:** before applying a non-stale event, run a fake external verification step using `fixtures/provider-plan.json`. Default is success. Retry `429` and `503`; treat `422` as a permanent failure. Allow at most three processing attempts in the normal provider-failure path, including the initial attempt. Schedule increasing, configurable backoff; tests may use short delays or a controlled clock. Do not busy-loop. Document whether/how a crashed worker consumes an attempt and why.
7. **Failure visibility:** after a permanent failure or exhausted retries, retain a terminal failed event with attempt history and a useful error. Do not change the job projection for failed verification. A lower version may remain visible if a higher version fails verification; a later successful higher version can replace it. Lower/equal stale work may skip verification and finish as a no-op.

Implement these read endpoints, or document a clearly equivalent contract:

- `GET /jobs?tenantId=...&sourceId=...&status=active&limit=20&cursor=...`: tenant-scoped list, default active, capped page size, deterministic pagination. Support `archived` and `all` for inspection. Define behavior when data changes between pages.
- `GET /events/:eventId?tenantId=...&sourceId=...`: acceptance/processing state, attempt count and last error. A duplicate response should allow the caller to find this existing event.
- `GET /health`: basic service/dependency readiness. Give your documented status names a clear meaning.

The contract specifies outcomes, not a required collection layout or locking algorithm. Explain your chosen data model, indexes and atomicity boundaries. Do not claim exactly-once processing merely

because duplicate API requests are rejected.

Demo and tests

Provide one documented setup command and one demo command. The demo must run the supplied scenario in fixture order, wait for phase 1 to settle, then submit the delayed phase 2 events and wait again. HTTP status is checked at submission; settled job/event outcomes are checked after draining. The fixtures contain 20 requests, including invalid inputs, replays, tenant/source collisions, stale changes and failures. Do not hardcode fixture IDs into business logic.

Include executable checks for:

- Validation, normalization, duplicate replay, conflicting reuse, and corrected reuse after a rejected request.
- Tenant/source isolation and out-of-order version processing, including archive-before-upsert and delayed stale updates.
- Retry success, permanent failure and exhaustion with visible attempts/backoff.
- Concurrent duplicate requests and concurrent workers.
- A deliberate crash/stop after writing a job projection but before acknowledging work, followed by recovery. Show why the final state is correct and there is no duplicate logical job; document any limits on external side effects.
- Persistence across API/worker restart and list pagination boundaries.

Use a real MongoDB integration test for unique indexes, version races, claims and recovery; mocks alone cannot validate database concurrency. Testing tools are your choice. Prioritize risky cases and identify missing coverage honestly.

Reasoning and scalability deliverables

Write `DESIGN.md` before or alongside implementation. Explain invariants, work lifecycle, atomicity, collection/index choices and at least two alternatives you rejected. Discuss what happens if acceptance, projection updates or completion acknowledgments fail independently. Identify your first likely bottleneck and one tradeoff you deliberately accepted within the time budget.

Write `SCALE.md` for growth from **100,000 events/day to 10 million/day**, with bursts of **5,000 events/second**, a target of processing accepted events within **5 minutes under the stated design capacity**, 1 KB average raw event payload, 7-day raw event retention, and roughly 1 million current job projections. State your own processing-cost, index, storage and replication assumptions. Include:

- Average versus burst arrival rates, worker capacity and backlog/drain-time calculations. State how long your design can sustain the burst; a peak rate without a duration is incomplete.
- MongoDB indexes, retention/TTL, hot keys, write contention and a possible partition/shard strategy with its uniqueness constraints and tradeoffs.
- Backpressure/admission control, noisy-tenant fairness, retry storms and bounded resource use.
- Metrics and alerts for processing lag, queue age, retries/failures and database saturation. Explain when you would add a separate broker or change the architecture.

Run a reproducible local load scenario with **at least 1,000 distinct valid events, plus 200 exact replay requests**, including at least 50 jobs with versions submitted out of order. Use separate load-test tenant IDs. Record machine/configuration, concurrency, accepted/replay/error counts, HTTP p50/p95 latency, final-state checks and queue drain time. Report what you actually measured; the production numbers are a design discussion, not a required laptop benchmark. Do not extrapolate linearly without stating the assumptions.

AI use and mandatory post-generation QC

Submit `AI_USAGE.md` and `QC_REPORT.md` using the supplied templates. Use AI on at least one meaningful artifact, then review it yourself. Record the tool/model if known, a short prompt excerpt or summary, the artifact/commit, what you accepted or changed, and how you verified it. Full chat history and private information are unnecessary. If you found no material defect, say so and show the independent checks; do not invent mistakes.

The final QC pass must happen **after** the last AI-generated code or document change. It must cover generated and handwritten changes alike:

- Run formatting/linting, TypeScript checking, unit/integration tests, clean setup/demo and the load scenario. Record the actual commands, results and environment; mark anything not run or failing.
- Challenge at least three plausible failure hypotheses, including one concurrency/recovery case and one documentation claim. Record evidence even if the checks pass.
- Compare README/API examples, indexes, retry settings, diagrams and benchmark claims with the final code and observed behavior. Correct unsupported claims.
- Review untrusted input handling, tenant scoping, credentials and dependencies. Do not commit credentials or raw AI conversation secrets.
- Tie evidence to a **code commit SHA** and identify the **final submission SHA**. QC documents can be committed afterward if code is unchanged. If code or generated documentation changes again, rerun the affected checks and update the report. An AI review alone is not proof: you must run and interpret the relevant checks.

Git submission and short walkthrough

Submit the repository URL, final commit SHA, setup/demo commands, and any known limitations. A public repository is optional; for a private repository arrange reviewer access. Include source, lockfile, Docker Compose, `.env.example`, tests, fixtures, scripts, and these documents: `README.md`, `DESIGN.md`, `SCALE.md`, `AI_USAGE.md`, `QC_REPORT.md`. Keep meaningful progress commits, including an early design note and the final QC record. A `DEMO.md` with expected commands/results or a short recording makes the working demo easy to review. Deployment to a paid service is not required.

Be ready for a **25-minute walkthrough**: run the demo, explain one race/failure path, defend one scaling calculation, show one AI-assisted change and its QC, and make a small change with a regression test. AI tools may remain available; you must explain the decisions and evidence in your own words.